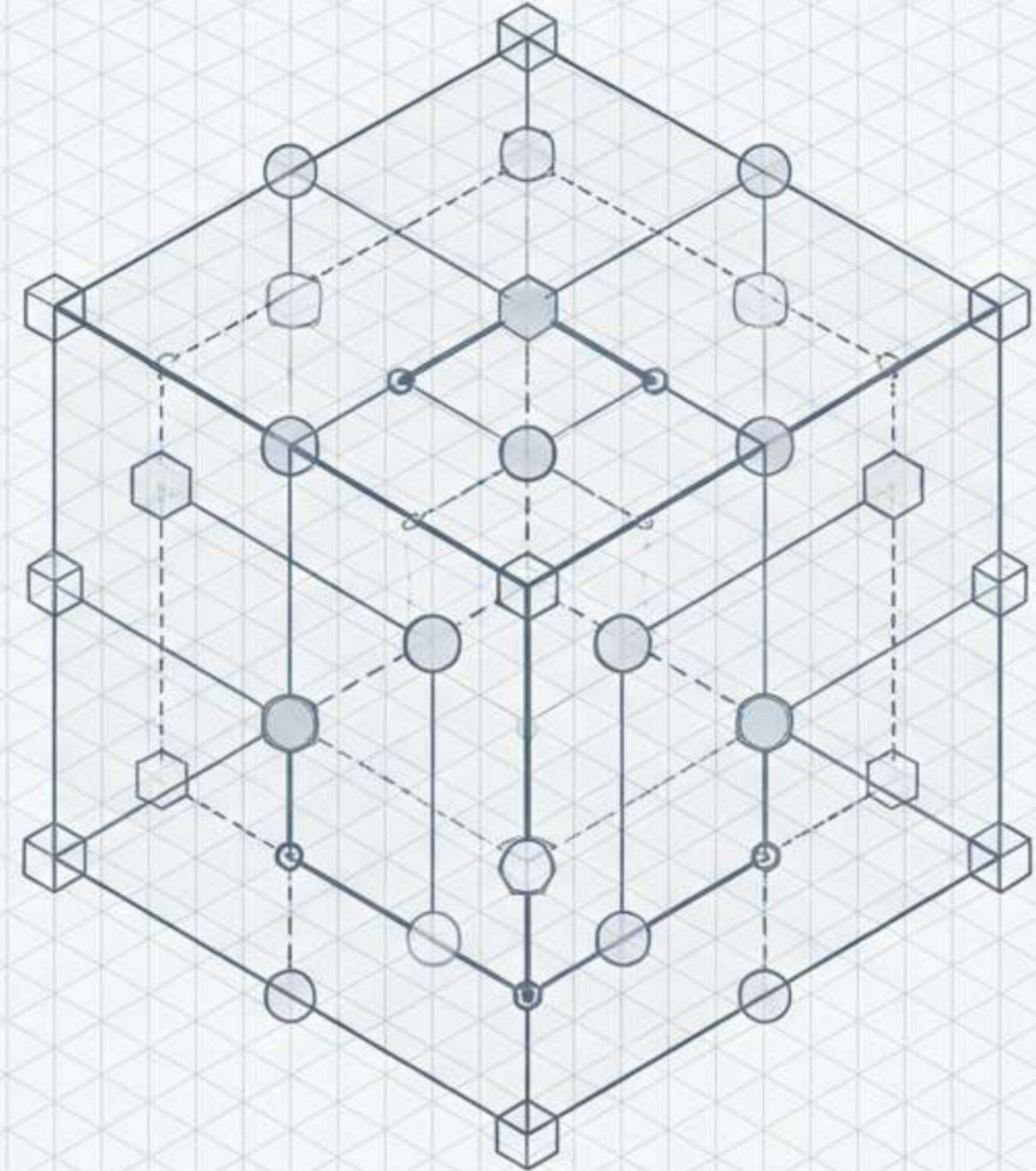
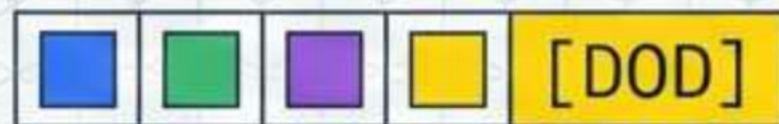


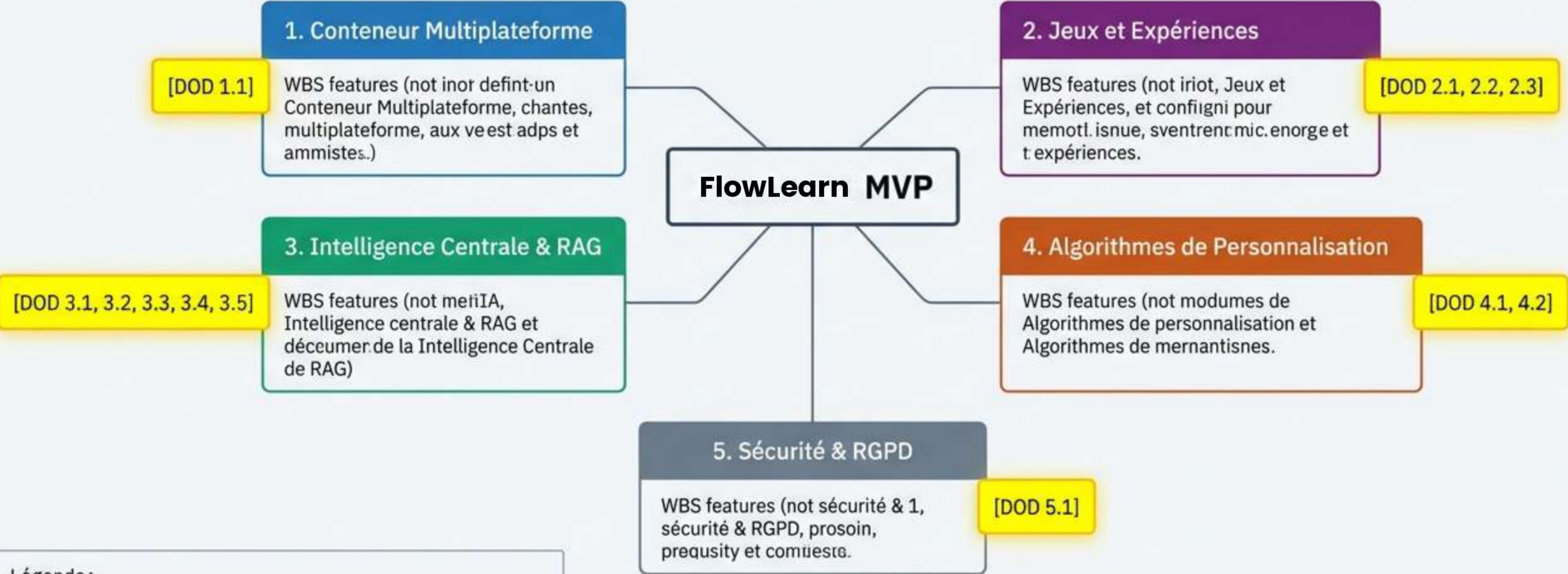
Le Plan d'Ingénierie FlowLearn MVP (v0.1)

Architecture système, Structure de Répartition du Travail (WBS) et Critères d'Acceptation (DOD) pour le déploiement du MVP.



Cartographie Globale du Produit (PBS) et de sa Validation (DOD)

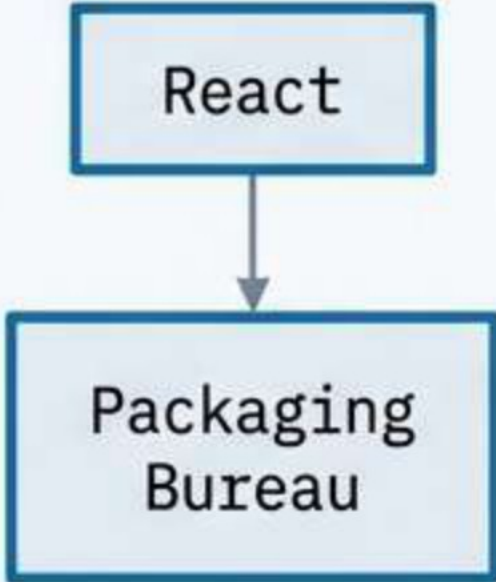
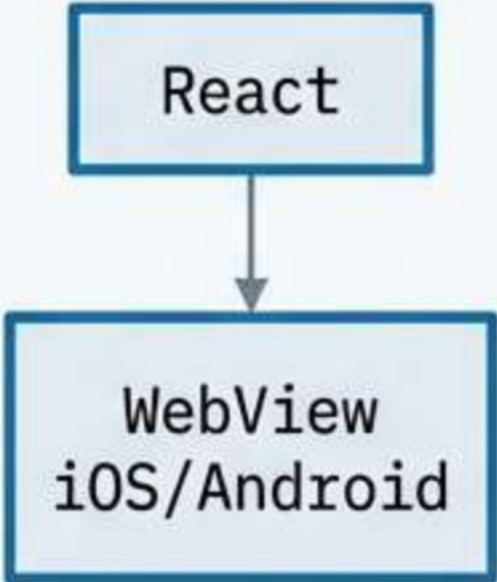
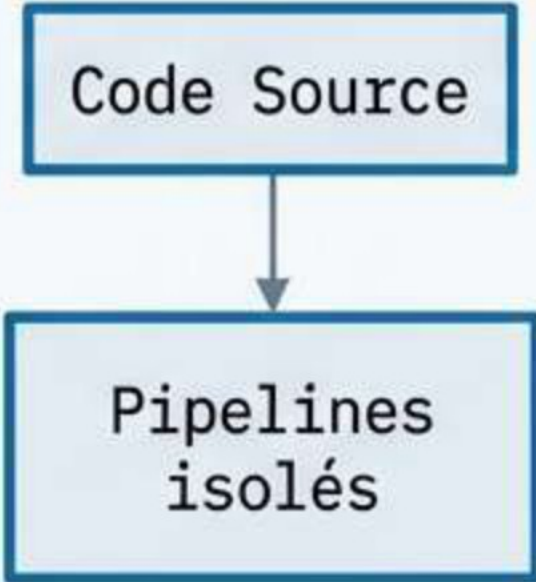
Une vue macroscopique des cinq piliers de FlowLearn. Chaque composant développé (WBS) est strictement encadré par un contrat de validation (DOD).



Légende:
Carrés Jaunes = Points de validation stricts (DOD)

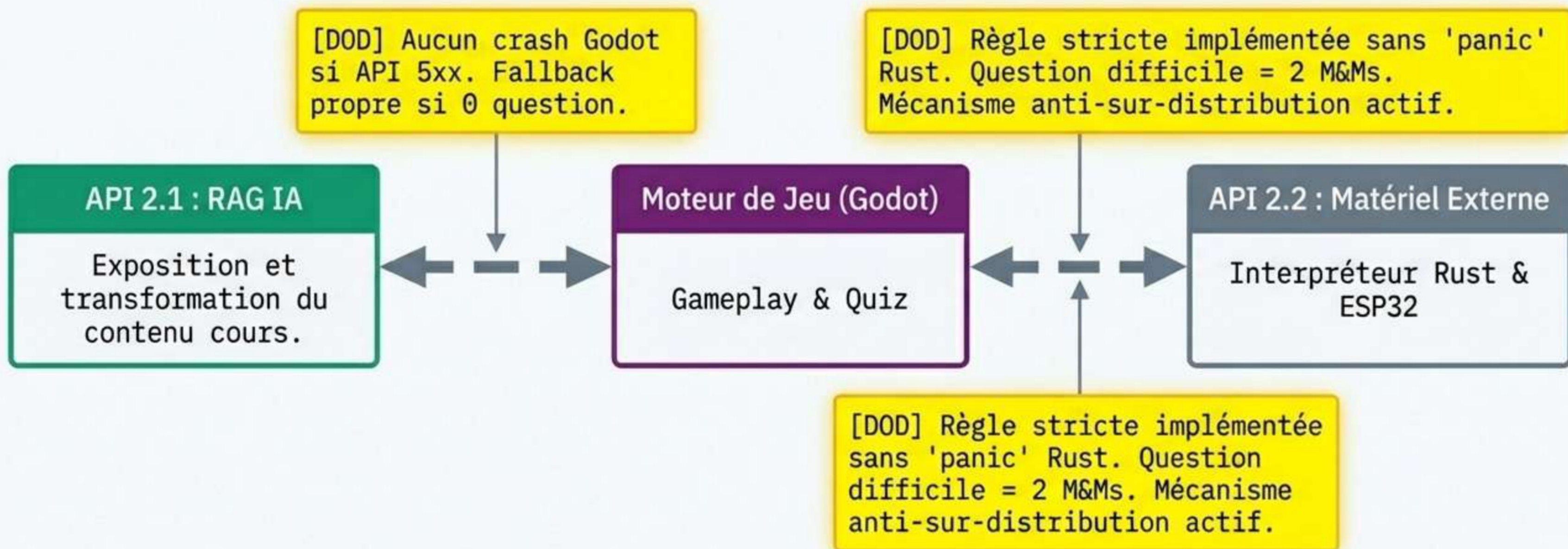
[Pilier 1] Déploiement Multiplateforme : Du React au Natif

PBS : Adapter l'application React web en logiciels bureau et mobile à partir d'un code source unique.

Electron (1.1.1)	Capacitor (1.1.2)	Build & Livraison (1.1.3)
 <pre>graph TD; React[React] --> PB[Packaging Bureau];</pre>	 <pre>graph TD; React[React] --> WV[WebView iOS/Android];</pre>	 <pre>graph TD; CS[Code Source] --> PI[Pipelines isolés];</pre>
[DOD] L'app s'ouvre sans écran blanc ; les builds sont reproductibles via documentation.	[DOD] Aucune erreur CORS ; réseau et appels API fonctionnels sur émulateur/appareil.	[DOD] Le build d'une plateforme ne casse jamais les autres.

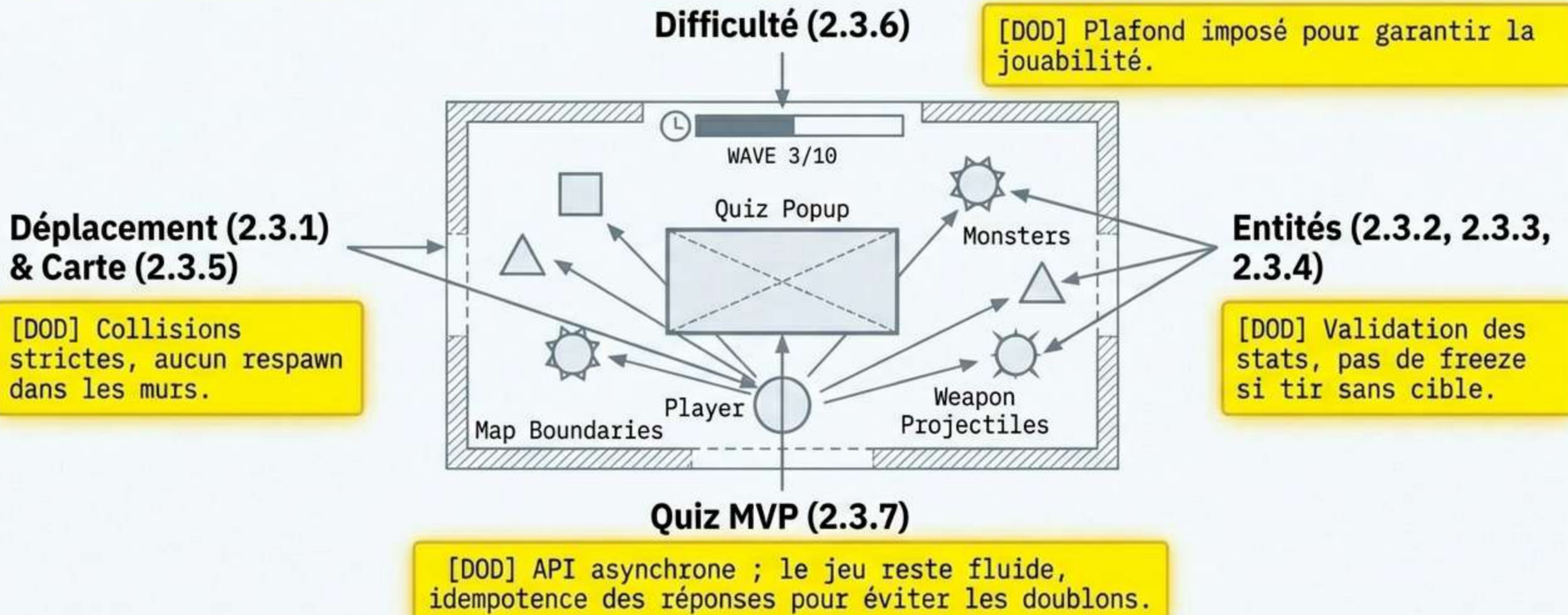
[Pilier 2] Les Ponts API : Connecter l'IA au Jeu, et le Jeu au Réel

PBS : Récupérer le cours (RAG) pour le gameplay, et traduire les réponses en récompenses physiques (IoT).



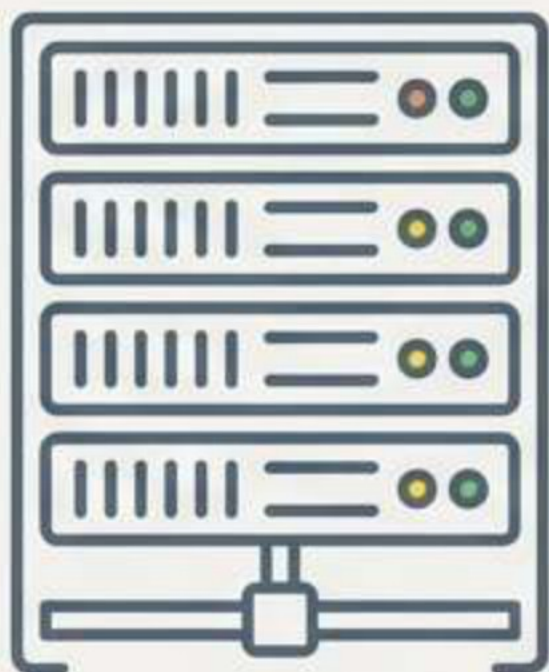
[Pilier 2] Anatomie du Gameplay : Survie et Quiz Intégré (2.3)

PBS : Créer un jeu de survie casual où le quiz cognitif est la mécanique centrale de récompense.



[Pilier 3] L'Infrastructure Centrale : Serveur Asynchrone et Base Vectorielle

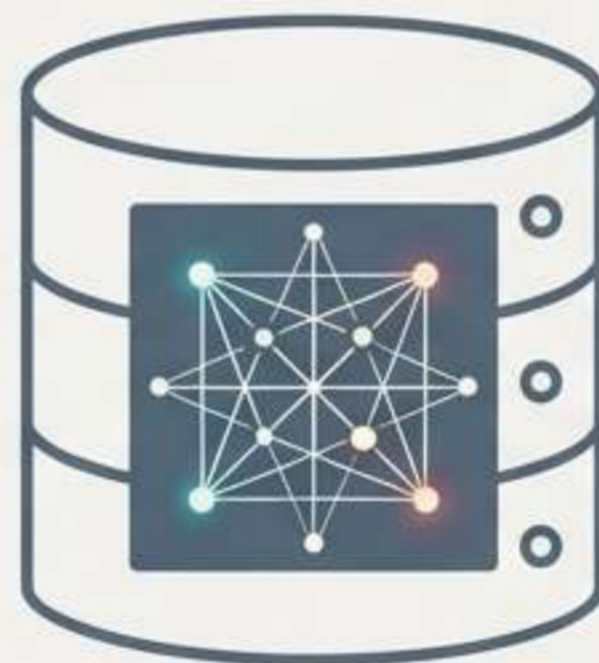
PBS : Assurer la communication hautes performances (FastAPI) et le stockage optimisé des embeddings (pgvector).



Serveur FastAPI (3.1)

Structure modulaire, endpoints d'ingestion/quiz, robustesse opérationnelle.

[DOD] Démarrage protégé avec validation des variables d'environnement. Rejet des entrées via erreurs 422/400 claires (validation Pydantic).



Base de Données Vectorielle (3.2)

Instance Supabase, schéma relationnel (documents, chunks, embeddings).

[DOD] Aucun secret en dur. Schéma multi-utilisateurs préparé pour l'anonymisation RGPD. Migrations versionnées.

[Pilier 3] Le Bibliothécaire : Pipeline d'Ingestion RAG (3.3)

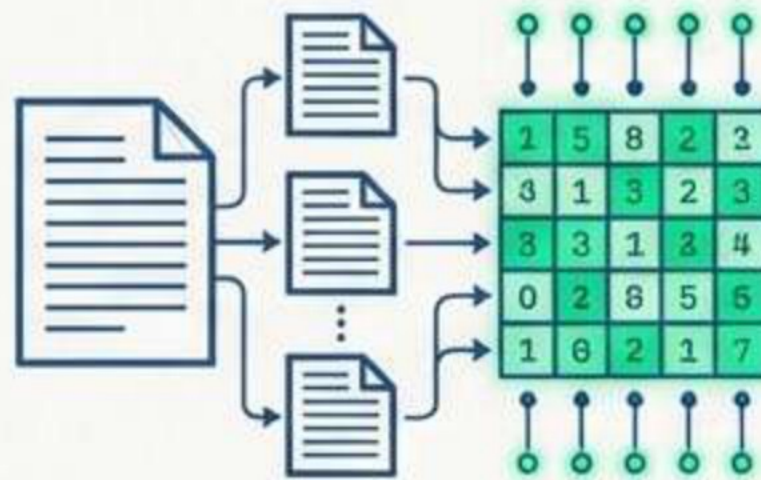
PBS : Importer les cours, les découper sémantiquement, et exposer un moteur de recherche par similarité.

1. Ingestion LlamaIndex



[DOD] Fichiers corrompus ou 'images seules' rejetés sans crash backend.

2. Chunking & Embeddings



[DOD] Stratégie de ré-ingestion déterministe (upsert propre). Chunks traçables à la source.

3. Retrieval (K-NN)



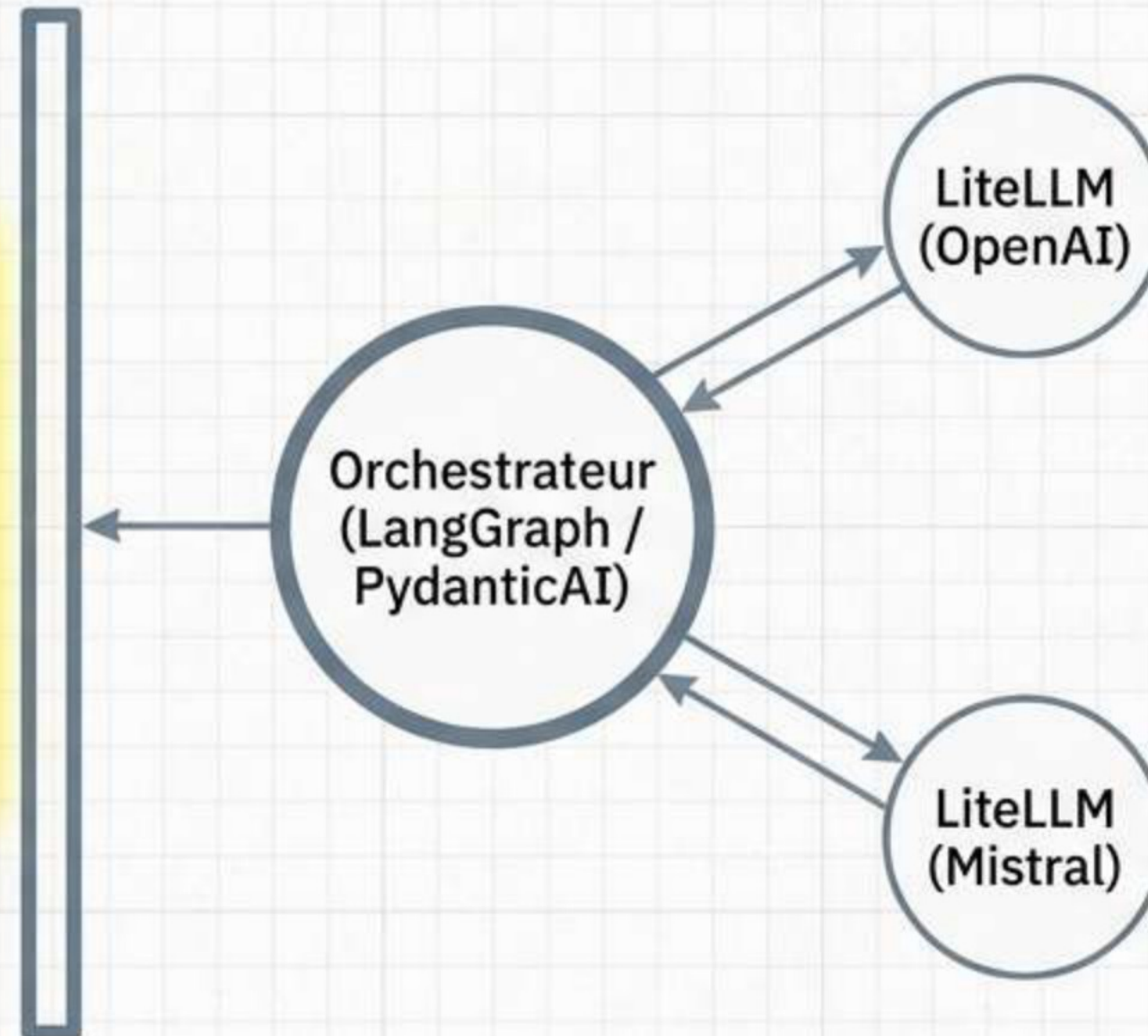
[DOD] Contrat Pydantic stable renvoyé à l'orchestrateur. Aucune exception fatale si la base est vide.

[Pilier 3] Orchestration IA : Génération Validée et Contrôlée (3.4 & 3.5)

PBS : Piloter les flux IA de génération de quiz (LiteLLM) avec une validation stricte des sorties (PydanticAI).

Le Bouclier Pydantic (3.4.1)

[DOD] REJET IMMEDIAT de toute sortie LLM invalide, partielle, ou mal typée.
Aucune donnée mal formée ne se propage vers Godot.

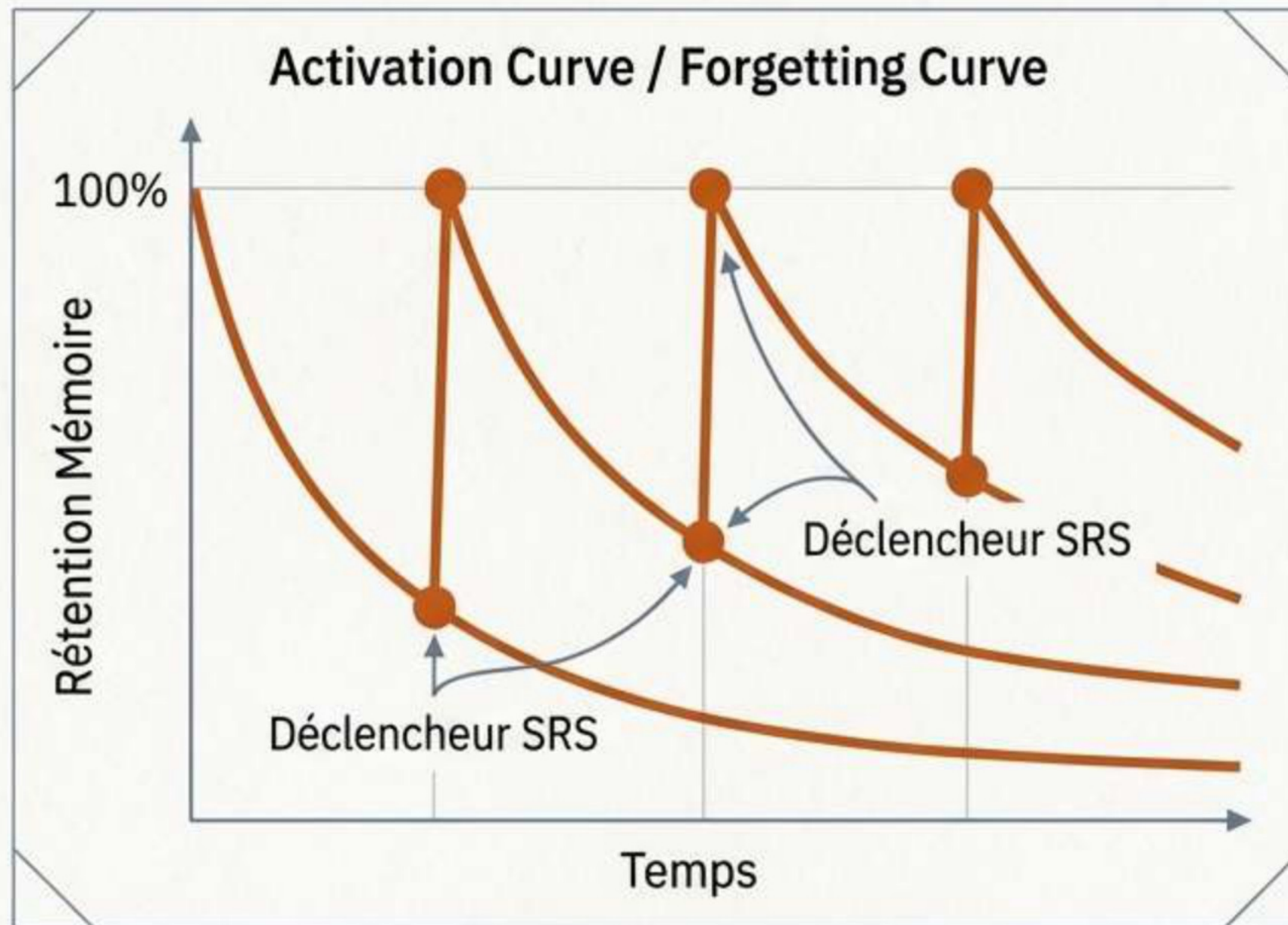


[DOD] Retry avec backoff sur timeout 5xx. Gestion claire des dépassements de quota sans crash.

[DOD] Retry avec backoff sur timeout 5xx. Gestion claire des dépassements de quota sans crash.

[Pilier 4] Sélection Intelligente : L'Algorithme SRS (4.1)

PBS : Sélectionner la prochaine question à poser via une logique de répétition espacée (type Anki).



Modèle de Données (4.1.1)

[DOD] Rejet des dates incohérentes (futur) par le backend.

Logique Anki (4.1.2)

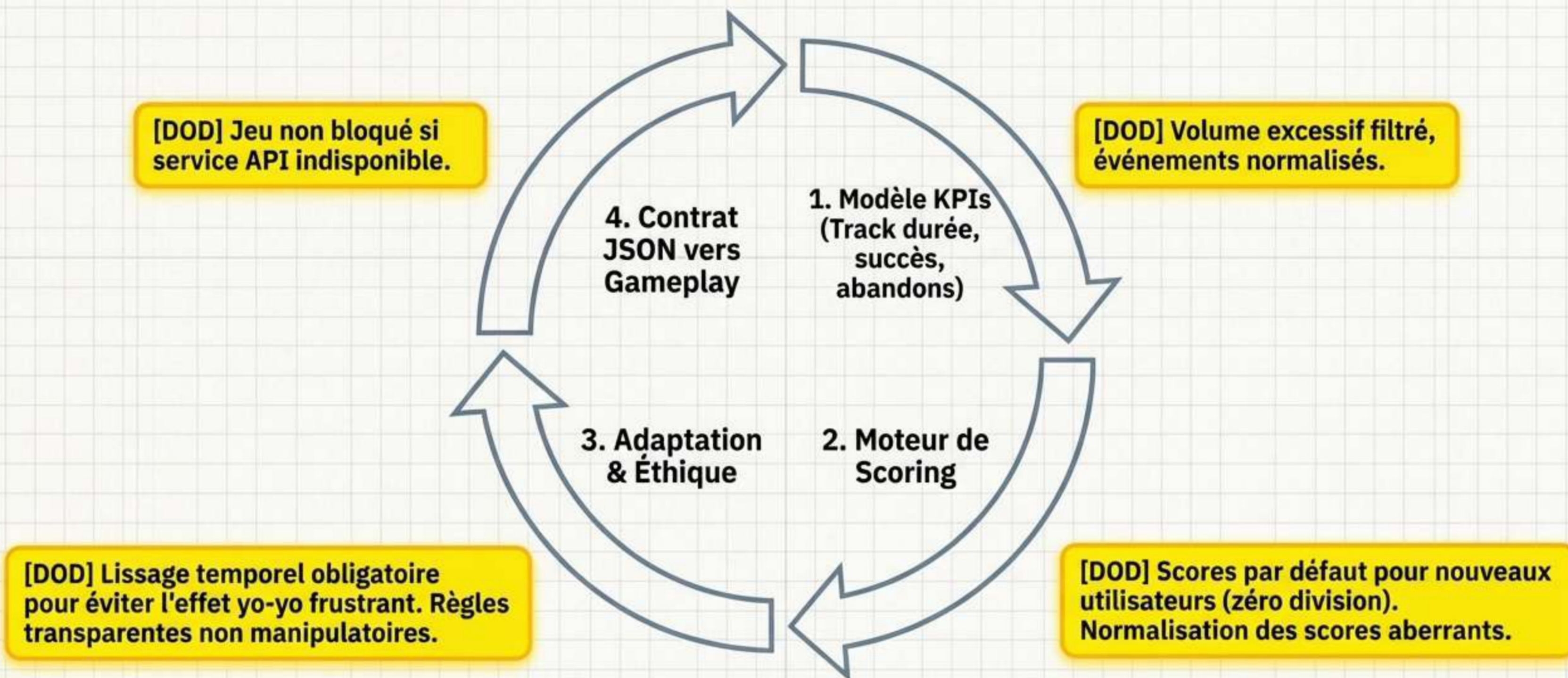
[DOD] Bascule propre sur de nouvelles cartes si aucune n'est due ; pas de boucle infinie.

Endpoint 'Next Question' (4.1.3)

[DOD] Renvoie un JSON standardisé. Évite la répétition abusive en s'appuyant sur l'historique utilisateur.

[Pilier 4] Moteur de Ciblage : La Boucle de Rétention Hook (4.2)

PBS : Analyser les KPIs pour adapter dynamiquement la difficulté du jeu, de manière éthique.



[Pilier 5] Le Coffre-Fort : Protection RGPD & Cybersécurité (5.1)

PBS : Garantir la sécurité de l'architecture et l'anonymisation totale des données d'apprentissage.

[Q1: Accès & Secrets (5.1.2)]

[DOD] Tolérance Zéro. **AUCUN** secret (API, DB) en dur dans le code. **100% Variables d'environnement/Vault.**

[Q2: Consentement (5.1.1)]

[DOD] Mentions légales à jour et processus d'exercice de droits documenté.



[Q3: Suppression & Rétention (5.1.3)]

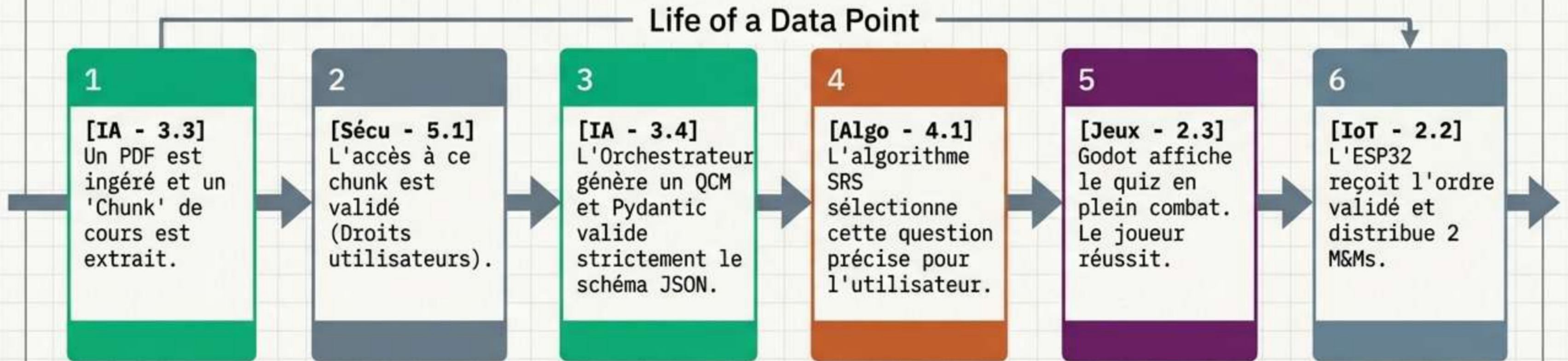
[DOD] Suppression en cascade testée (compte, chunks, KPIs). Aucune donnée orpheline laissée en base.

[Q4: Audit (5.1.4)]

[DOD] Toutes vulnérabilités critiques identifiées par le Pôle Cybersécurité sont corrigées ou documentées.

Synthèse : Le Flux de Données Immuable

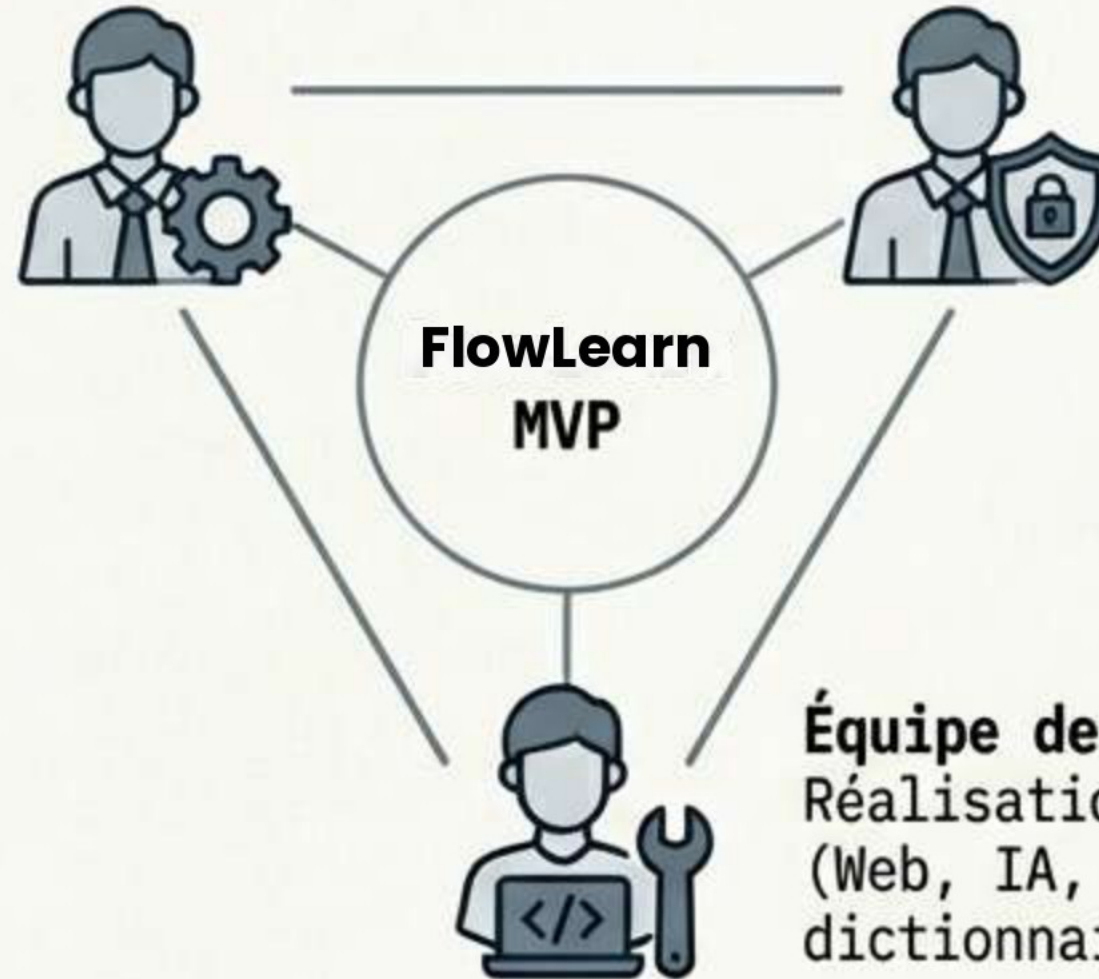
Le **Contrat JSON** (Livrable 2.3) est le ciment de l'architecture. Voici comment les 5 piliers collaborent sans friction pour une seule question :



Alignement des Responsabilités pour le MVP

La vision est documentée, les composants sont cartographiés, les critères d'acceptation sont fixés.

Porteur de Projet :
Garant de la vision stratégique et des règles (ex. algorithmes éthiques).



Pôle Cybersécurité :
Garant de la robustesse (Security by design, audits, RGPD).

Équipe de Développement :
Réalisation technique multidisciplinaire (Web, IA, Jeux, IoT) et respect du dictionnaire PBS.

Takeaway: Chaque ligne de code écrite doit servir une Feature (WBS) et valider un Contrat (DOD).